

10주차 발표 요약본

발표 주제: 토큰화와 임베딩

토큰화 개요 (5.0)

- 자연어(Natural Language)는 인간이 사용하는 언어이며, 자연어 처리(NLP)는 이를 컴퓨터가 이해하고 해석·생성하도록 하는 인공지능 기술
- 컴퓨터는 자연어를 직접 이해할 수 없으므로 말뭉치를 토큰 단위로 분리하는 과정이 필요하다. 이 과정을 토큰화 라 하며, 이를 수행하는 알고리즘 또는 도구를 토큰나이저 라 한다.
- 토큰화 방법에는 공백 분할, 정규표현식, 어휘 사전 기반, 머신러닝 기반 등이 있고, 특히 사전 기반의 경우 사전에 없는 단어(OOV, Out-Of-Vocabulary)가 존재할 수 있어 이를 고려해야 한다.
- 토큰나이저의 선택은 모델 성능에 큰 영향을 미친다.

단어 및 글자 토큰화 (5.1)

- 단어 토큰화: 문장을 단어 단위로 분리하는 방식.
 - 품사 태깅, 개체명 인식, 기계 번역 등에서 사용되며 가장 기본적인 전처리 방법
- 글자 토큰화: 글자 단위로 분리하여 더 작은 어휘 사전을 만들 수 있음.
메모리 효율이 높고 OOV 문제를 줄이지만, 의미 단위가 사라져 문맥 파악이 어려워질 수 있다.
- 자모 토큰화: 한글을 자모 단위로 분리하여 접사나 문장부호 의미까지 학습 가능하지만, 토큰 단위의 의미 결합이 필요하고 계산량이 늘어난다.

형태소 토큰화 (5.2)

형태소는 의미를 가지는 최소 단위이며, 형태소 토큰화는 이를 기준으로 문장을 분리한다.

한국어는 교착어이므로 형태소 단위 분석이 중요하다.

- 자립형태소: 의미를 스스로 가지는 형태소 (예: 명사)
- 의존형태소: 다른 형태소에 의존해야 의미가 형성됨 (예: 조사, 어미)

KoNLPy 라이브러리

- 한국어 형태소 분석 라이브러리로, Okt, Kkma, Komoran 등을 지원
- 분석기마다 결과가 달라 목적에 맞게 선택해야 함

NLTK

- 영문 자연어 처리를 위한 라이브러리로, 토큰화·품사 태깅·감성 분석 등 다양한 기능을 제공
- 대표 모델로 Punkt(통계 기반) 과 Averaged Perceptron Tagger(퍼셉트론 기반) 이 있다.

spaCy

- 사이언 기반 오픈소스 라이브러리로 빠른 속도와 높은 정확도를 목표로 한다.
- NLTK가 교육용이라면 spaCy는 실제 산업 환경용으로 적합하다.

하위 단어 토큰화 (5.3)

- 기존 형태소 분석의 한계를 해결하기 위해 제안된 방법으로, 단어보다 작고 문자보다 큰 서브워드 단위를 사용한다.
- 장점 : OOV 감소, 어휘 축소, 형태 변화 대응(어미/접사)

BPE (Byte Pair Encoding)

1. 문자 단위로 시작하고 자주 등장하는 인접 쌍을 병합하는 방식
2. 빈도 기반의 병합 규칙(merges)과 어휘집(vocab)을 생성
3. 간단하고 효율적이며 신어 대응이 뛰어나다
 - 예: "lowest" → ["low", "est"]

SentencePiece & Korpora

- SentencePiece: 구글에서 개발한 하위 단어 토큰라이저로 BPE와 유사.
 - 언어 비의존적이며, 공백 없이 raw text로 직접 학습 가능.
 - BPE / Unigram 알고리즘 지원, 특수 토큰 및 정규화 자동 처리.
- Korpora: 한국어 공개 말뭉치 수집 라이브러리(예: 네이버 영화평).
 - 실습 시 쉽게 corpus.txt 파일을 생성하고 BPE 모델을 학습할 수 있다.

WordPiece

- BERT에서 사용되는 알고리즘으로 확률 기반(우도 최대화) 병합 방식을 사용
- BPE가 단순 빈도 기반이라면, WordPiece는 문장 확률을 높이는 조합을 어휘에 포함시킨다.
- 서브워드는 보통 "##" 접두사를 가짐

Hugging Face tokenizers

- Rust 기반 고성능 토크나이저로 BPE, WordPiece, Unigram 등 다양한 방식을 지원
- 정규화·프리티크화·포스트프로세싱 단계를 모듈화하며, Transformers 라이브러리와 호환됨
- WordPiece 학습 시 공백 의존도를 낮추기 위해 ByteLevel/Metaspace 방식을 사용할 수 있다.

임베딩 (6.0~6.3)

6.0 임베딩 개요

- 컴퓨터는 텍스트를 직접 이해할 수 없으므로 벡터화(Text Vectorization) 필요
 - One-Hot Encoding: 각 단어를 고유 인덱스로 매핑하여 단순 표현
 - Count Vectorization: 단어 빈도를 세어 벡터로 표현
 - 단점: 벡터 희소성(Sparsity), 문맥 정보 손실, 의미 반영 어려움

워드 임베딩(Word Embedding)

- 단어를 고정된 실수 벡터로 표현하는 방법.
- 대표적 기법: Word2Vec, fastText
- 의미적 유사성을 학습하지만, 고정 임베딩이라 다의어나 문맥 반영이 어렵다 → 동적 임베딩 필요

6.1 언어 모델

문장의 확률을 계산해 문맥을 예측하는 모델로, 번역·음성 인식·요약 등에 활용

- 자기회귀 언어 모델: 이전 단어를 기반으로 다음 단어를 예측
 - 조건부 확률과 연쇄법칙을 이용해 문장 전체 확률을 계산
- 통계적 언어 모델: 마르코프 체인 기반 빈도 통계로 문맥을 추정하지만 데이터 희소성 문제가 존재한다.

6.2 N-gram

- N개의 - 연속된 단어 시퀀스를 하나의 단위로 처리하는 단순한 통계적 모델
- 유니그램(N=1), 바이그램(N=2), 트라이그램(N=3) 등으로 구성되며, 작은 데이터셋의 패턴 분석이나 관용어 추출에 사용

6.3 TF-IDF

- 문서 내 단어의 중요도를 계산하는 통계적 기법으로, TF(단어 빈도) 와 IDF(역문서 빈도) 의 곱으로 계산됨
- TF 높음 → 해당 문서에서 중요
- DF 높음 → 일반적인 단어 (중요도 낮음)
- IDF는 희귀 단어에 높은 가중치를 부여
 - TF-IDF는 단어의 중요도를 표현하지만 문맥과 의미를 반영하지 못함
- 활용 방법
 1. TfidfVectorizer 객체 생성
 2. fit()으로 단어 사전 학습
 3. transform()으로 TF-IDF 벡터 변환
 4. fit_transform()으로 학습+변환 동시 수행
 - 주요 매개변수: lowercase, stop_words, ngram_range, max_df, min_df, smooth_idf 등